

Yıldız Technical University

Computer Engineering

BLM3120 Information Retrieval and Web Search Engines

Term Project

Prof. Dr. Mehmet Sıddık AKTAŞ

Multi-Class Text Classification of E-commerce Products

21011021 – İsmail İPEK

21011022 – Tarık TAŞKIN

1. Project Description

In the field of Information Access and Search Engines, e-commerce platforms represent a highly important example of domain-specific search systems. Unlike general-purpose search engines, e-commerce search engines must operate on structured and semi-structured product data while also effectively handling unstructured textual content such as product titles and descriptions.

One of the most critical factors affecting retrieval quality in e-commerce platforms is the accuracy and consistency of product metadata, particularly product categories. Categories play a central role in search, filtering, ranking, and recommendation systems. However, in large-scale e-commerce environments, manually assigning categories to products is time-consuming, error-prone, and not scalable.

For this reason, automatic product categorization based on textual descriptions has become a fundamental problem in information access systems. Given a product's description, the system must infer the most appropriate category so that the product can be correctly indexed and retrieved during user searches. Inaccurate categorization may lead to irrelevant search results, poor user experience, and reduced retrieval effectiveness.

The purpose of this project is to design and implement a text-based product classification system that automatically predicts product categories using machine learning techniques. The project focuses on transforming raw product descriptions into meaningful numerical representations using TF-IDF vectorization, and training multiple supervised classification models to perform category prediction.

By integrating data preprocessing, machine learning-based classification, evaluation metrics, and an interactive user interface, this project demonstrates how information retrieval concepts can be practically applied to domain-specific search problems in e-commerce systems.

2. Dataset

The dataset used in this project consists of e-commerce product information and is designed to support a domain-specific text classification task. Each data instance represents a single product and includes a free-text product description along with a corresponding category label. The textual descriptions contain detailed information about products, such as features, usage scenarios, and specifications, while the category labels indicate the high-level product group to which each item belongs. The dataset includes four distinct categories: **Books, Clothing & Accessories, Electronics, and Household**. These categories are commonly used in e-commerce platforms and provide a suitable basis for evaluating automatic product categorization in the context of information access and search engines.

Before model training, the dataset was analyzed in terms of size and data quality. The original dataset contained missing values and duplicate records, which are common issues in real-world e-commerce data. To improve the reliability of subsequent analyses and classification models, rows with missing product descriptions or category labels were removed, and duplicate entries were eliminated. After these cleaning steps, the dataset was reduced to a set of unique and valid samples that accurately represent the underlying product space. This preprocessing stage ensures that the models are trained on consistent and noise-reduced data.

After cleaning, the distribution of samples across categories was examined to understand the structure of the dataset. The cleaned dataset contains the following number of samples per category, as shown in **Table 2.1**.

Table 2.1: Number of Samples per Category

Category	**Number of Samples**
Household	10564
Books	6256
Clothing & Accessories	5674
Electronics	5308

The dataset shows a moderately balanced distribution, with Household being the largest category, followed by Books, Clothing & Accessories, and Electronics. Although the categories are not perfectly balanced, each class contains a sufficient number of samples to support supervised learning. This structure allows macro-averaged evaluation metrics to provide a meaningful assessment of model performance across all categories.

In addition to examining dataset size and category distribution, the textual properties of the product descriptions were analyzed to better understand the nature of the input data. Specifically, the average word count of product descriptions was computed for each category, both before and after the text cleaning process. The cleaning pipeline removes non-informative elements such as URLs, punctuation, numeric-only tokens, special characters, and excessive whitespace, resulting in more compact and standardized text representations.

The comparison of mean word counts before and after cleaning shows a consistent reduction across all categories, indicating that the preprocessing step successfully removes noise while preserving the semantic structure of the descriptions. Categories such as Books and Electronics tend to have longer descriptions, reflecting the more detailed information typically associated with these products, whereas Clothing & Accessories generally contains shorter descriptions. The effect of cleaning on word counts across categories is illustrated in **Figure 2.1**.

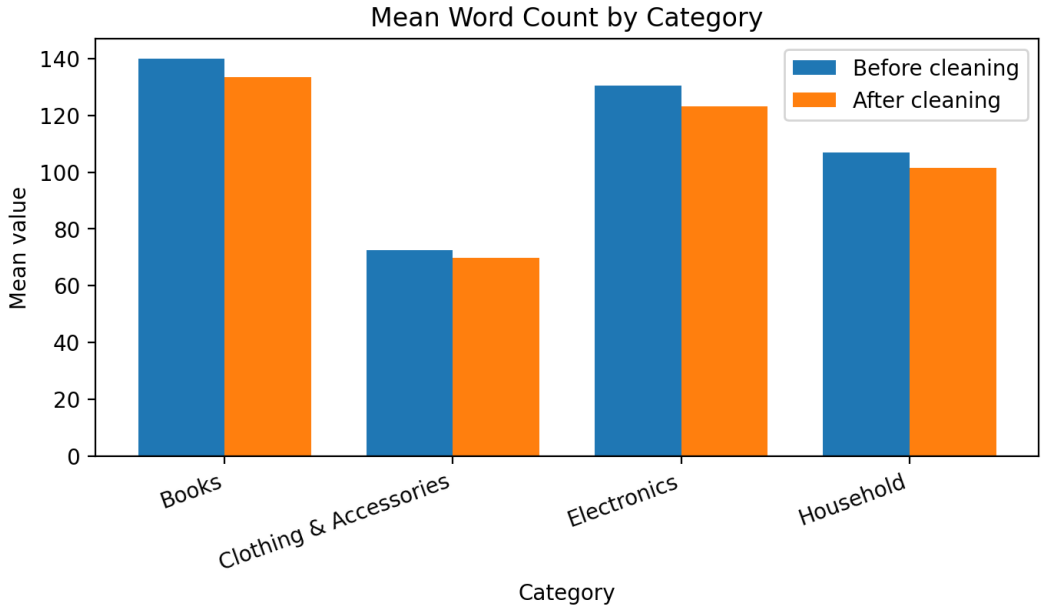


Figure 2.1: Mean Word Count by Category

A similar analysis was conducted for character counts. The average number of characters per product description decreases across all categories after cleaning, further confirming the effectiveness of the preprocessing pipeline. The consistent reduction in character counts indicates that the cleaning process simplifies the textual input without disproportionately affecting any single category. This behavior suggests that the preprocessing steps are applied uniformly and do not introduce category-specific bias. The character count comparison is presented in **Figure 2.2**.

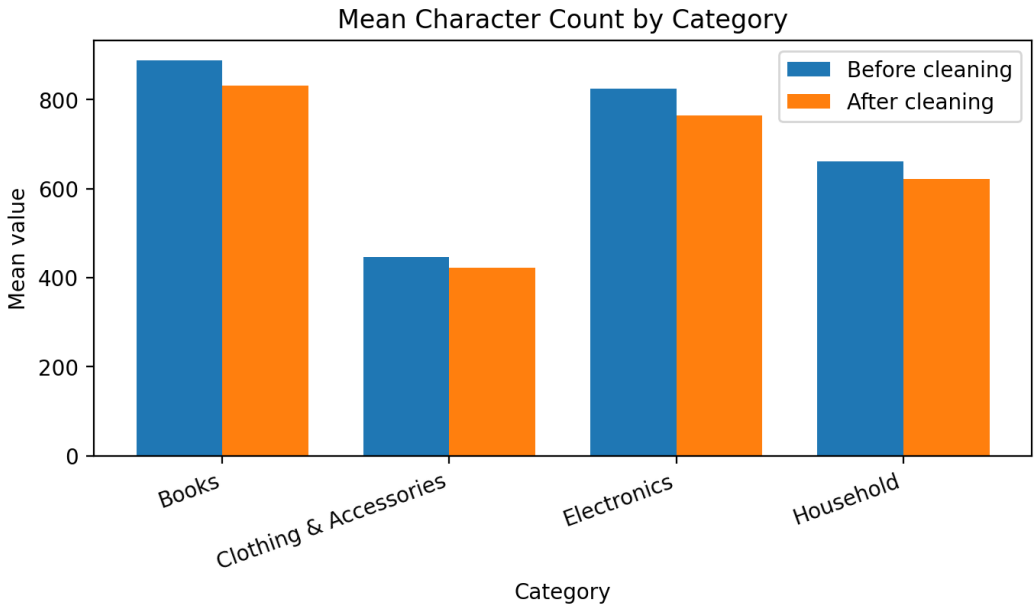


Figure 2.2: Mean Character Count by Category

Overall, the dataset represents a realistic and challenging example of e-commerce product data used in domain-specific information retrieval systems. Through systematic data cleaning and exploratory analysis, the dataset was transformed into a high-quality input suitable for machine learning-based classification. The analyses of category distribution and text length characteristics provide valuable insights into the structure of the data and form a solid foundation for the model training and evaluation stages of the project.

3. Algorithms and Hyperparameters

In this project, supervised machine learning methods were employed to automatically predict product categories based on textual descriptions. The overall classification pipeline consists of two main stages: transforming raw text into numerical feature representations and training classification models on these representations. Classical machine learning algorithms were deliberately chosen due to their interpretability, computational efficiency, and proven effectiveness in domain-specific text classification tasks commonly encountered in information retrieval systems.

Textual product descriptions were first converted into numerical vectors using the **Term Frequency–Inverse Document Frequency (TF-IDF)** representation. TF-IDF is a fundamental technique in information retrieval that quantifies the importance of a term within a document relative to its frequency across the entire corpus. This weighting scheme allows the models to emphasize discriminative terms while reducing the impact of commonly occurring but less

informative words. In this project, the TF-IDF vectorizer was configured to extract both unigrams and bigrams in order to capture not only individual keywords but also short word sequences that may convey additional semantic meaning. English stop words were removed to reduce noise, and a minimum document frequency threshold was applied to exclude extremely rare terms that could increase sparsity without contributing meaningful information to the classification task.

Following feature extraction, the dataset was divided into training and test sets using a stratified sampling strategy. Stratification ensures that the relative distribution of categories is preserved across both subsets, which is particularly important in multi-class classification scenarios. **70%** of the dataset was allocated for training, while the remaining **30%** was reserved for evaluation.

Model performance was assessed using multiple evaluation metrics, including **accuracy**, **macro-averaged precision**, **macro-averaged recall**, and **macro-averaged F1-score**. While accuracy provides a general measure of correctness, it can be misleading in the presence of class imbalance. Therefore, macro-averaged metrics were used to assign equal importance to each category, ensuring that performance is evaluated fairly across all classes. In addition to scalar metrics, **confusion matrices** were computed to analyze class-level prediction behavior and to identify patterns of misclassification between categories.

Logistic Regression

Logistic Regression is a linear classification algorithm that models the relationship between input features and class probabilities using a linear decision boundary. It is widely used in text classification tasks due to its simplicity, interpretability, and effectiveness when working with high-dimensional and sparse feature spaces such as TF-IDF representations.

In this project, Logistic Regression was selected as a baseline model because it provides a strong reference point for evaluating text classification performance. Its efficiency and stability make it well-suited for large-scale textual data, and its linear nature allows the model to generalize effectively without excessive computational cost. Additionally, Logistic Regression offers interpretable results, which is particularly valuable in information retrieval contexts where understanding model behavior is important. To address potential class imbalance in the dataset, balanced class weights were applied, and the maximum number of iterations (1000) was increased to ensure proper convergence.

The Logistic Regression model achieved strong overall performance on the test set, as shown in **Table 3.1**, with an accuracy of approximately **95%**. Macro-averaged precision, recall, and F1-score are all reported as **0.95**, indicating that the model performs consistently across all categories and does not favor any particular class. This balanced performance demonstrates that the model is effective at handling multi-class classification in a domain-specific e-commerce setting.

At the class level, the model shows particularly strong results for *Clothing & Accessories*, which achieves a recall of **0.98** and an F1-score of **0.97**, indicating that product descriptions in this category are identified with high reliability. The *Household* category also exhibits strong performance, with a recall of **0.96** and an F1-score of **0.95**, suggesting that home-related products are consistently classified correctly. The *Books* category achieves a recall of **0.93**, with minor misclassifications likely caused by overlapping descriptive language. Similarly, the *Electronics*

category shows stable performance with both precision and recall around **0.94**, despite potential lexical overlap with other categories.

Table 3.1: Classification Report of Logistic Regression

	Precision	Recall	F1-score	Support
Books	0.96	0.93	0.94	1877
Clothing & Accessories	0.97	0.98	0.97	1702
Electronics	0.93	0.94	0.94	1593
Household	0.95	0.96	0.95	3169
accuracy			0.95	8341
macro avg	0.95	0.95	0.95	8341
weighted avg	0.95	0.95	0.95	8341

The confusion matrix, shown in **Figure 3.1**, further supports these findings, as high values along the diagonal indicate a strong rate of correct predictions for all categories. Limited confusion is observed primarily between *Household* and other categories such as *Books* and *Electronics*, which can be attributed to shared contextual terms related to usage or lifestyle. However, these misclassifications occur at relatively low frequencies and do not significantly impact the overall effectiveness of the model.

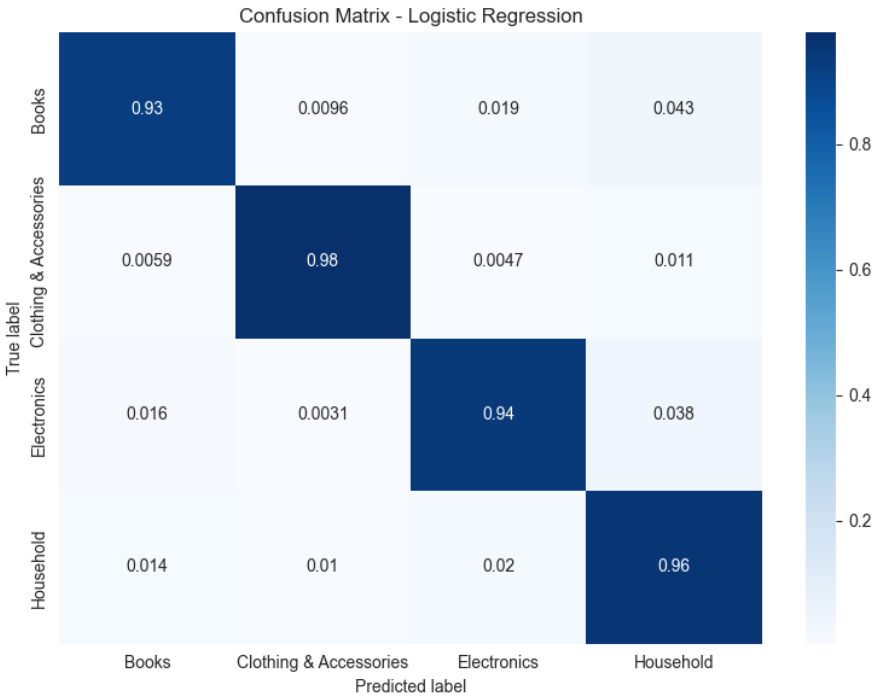


Figure 3.1: Confusion Matrix of Logistic Regression

Linear Support Vector Machine (Linear SVM)

The Linear Support Vector Machine is a discriminative linear classification algorithm that aims to find an optimal separating hyperplane by maximizing the margin between classes. Linear SVMs are particularly well-suited for text classification tasks, as they perform effectively in high-dimensional feature spaces generated by techniques such as TF-IDF and are robust against overfitting in sparse data settings.

In this project, the Linear SVM model was selected due to its strong empirical performance in document and text classification problems. Compared to probabilistic linear models, Linear SVMs often achieve better generalization by focusing on margin maximization rather than probability estimation. This makes them especially suitable for domain-specific classification tasks where feature dimensionality is high and class boundaries may be closely spaced. To mitigate potential class imbalance, balanced class weights were applied, ensuring that all categories contribute equally to the optimization process. A linear kernel was chosen to maintain computational efficiency and compatibility with sparse TF-IDF features.

The Linear SVM model achieved the highest overall performance among the evaluated models, with an accuracy of approximately **96%** on the test set, as shown in **Table 3.2**. The macro-averaged precision and F1-score are both reported as **0.96**, while the macro-averaged recall is **0.95**, indicating consistently strong performance across all categories. These results demonstrate that the model effectively balances precision and recall while maintaining robustness across classes.

At the class level, *Clothing & Accessories* again exhibits the strongest performance, achieving a precision, recall, and F1-score of **0.98**, indicating near-perfect classification for this category. The *Household* category also shows excellent results, with a recall of **0.96** and an F1-score of **0.96**, reflecting reliable identification of home-related products. The *Books* category maintains stable performance with a recall of **0.93** and an F1-score of **0.95**, while the *Electronics* category achieves balanced precision and recall values of **0.94**, demonstrating consistent classification despite overlapping terminology with other categories.

Table 3.2: Classification Report of Linear SVM

	Precision	Recall	F1-score	Support
Books	0.96	0.93	0.95	1877
Clothing & Accessories	0.98	0.98	0.98	1702
Electronics	0.94	0.94	0.94	1593
Household	0.95	0.96	0.96	3169
accuracy			0.96	8341
macro avg	0.96	0.95	0.96	8341
weighted avg	0.96	0.96	0.96	8341

The confusion matrix, shown in **Figure 3.2**, further confirms the strong performance of the Linear SVM model. High diagonal values indicate a high proportion of correct predictions for all categories. Similar to the Logistic Regression model, minor confusion is observed primarily between *Electronics* and *Household*, as well as between *Books* and *Household*, likely due to shared descriptive language related to usage context. However, these misclassifications remain limited and do not significantly affect overall model performance.

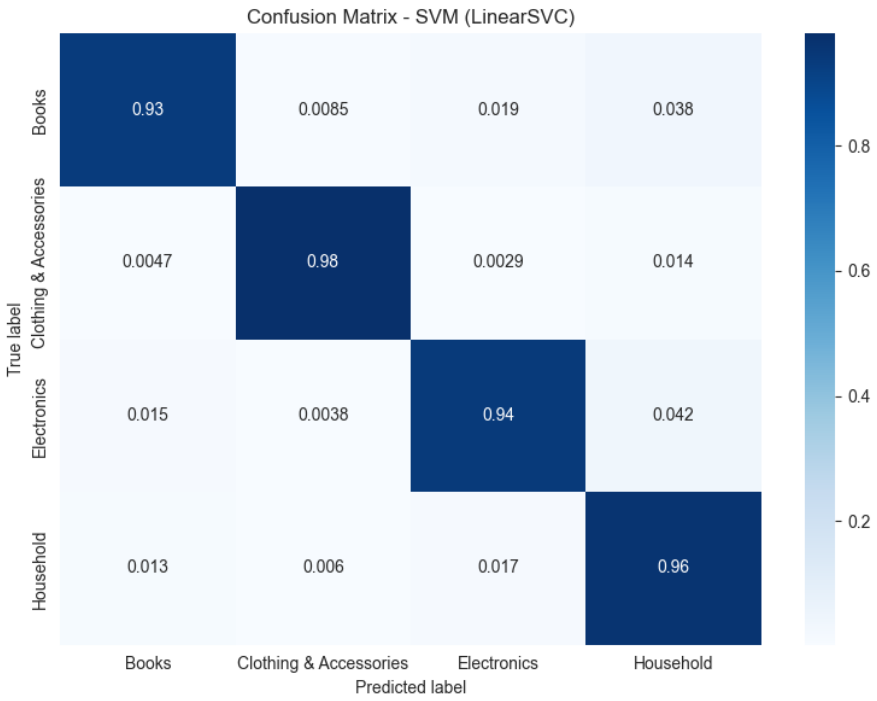


Figure 3.2: Confusion Matrix of Linear SVM

Random Forest

Random Forest is an ensemble-based machine learning algorithm that combines multiple decision trees to perform classification through majority voting. By aggregating the predictions of many individual trees, Random Forest models are able to capture non-linear relationships in the data and reduce variance compared to a single decision tree. This makes Random Forest a powerful and flexible classifier, particularly for problems where class boundaries are complex.

In this project, the Random Forest classifier was included to provide a non-linear baseline and to compare its performance against linear models such as Logistic Regression and Linear SVM. While Random Forests are not typically the first choice for very high-dimensional sparse text representations like TF-IDF, their inclusion allows for an evaluation of how ensemble-based methods perform in a domain-specific text classification setting. A fixed number of decision trees was used to balance computational efficiency and model capacity, and a random seed was specified to ensure reproducibility of the results.

Compared to the linear models, the Random Forest classifier achieved a lower but still competitive overall performance, with an accuracy of approximately **92%** on the test set, as shown in **Table 3.3**. The macro-averaged precision, recall, and F1-score are all reported around **0.92**, indicating a consistent but slightly weaker performance across categories when compared to Logistic Regression and Linear SVM. These results suggest that while Random Forest is capable of learning meaningful patterns from the data, it is less effective than linear classifiers for this particular text-based task.

At the class level, the *Clothing & Accessories* category again demonstrates strong performance, with precision, recall, and F1-score values of **0.95**, indicating that the model can reliably identify clothing-related product descriptions. The *Books* category achieves a recall of **0.91** and an F1-score of **0.92**, showing reasonably strong performance with some misclassification. The *Household* category maintains a recall of **0.94**, but precision drops to **0.90**, suggesting that some non-household items are incorrectly predicted as household products. The most notable performance drop is observed in the *Electronics* category, where recall decreases to **0.86**, indicating increased confusion with other categories.

Table 3.3: Classification Report of Random Forest

	Precision	**Recall**	**F1-score**	**Support**
Books	0.94	0.91	0.92	1877
Clothing & Accessories	0.95	0.95	0.95	1702
Electronics	0.92	0.86	0.89	1593
Household	0.90	0.94	0.92	3169
accuracy			0.92	8341
macro avg	0.93	0.92	0.92	8341
weighted avg	0.92	0.92	0.92	8341

The confusion matrix, shown in **Figure 3.3**, highlights these observations. While diagonal values remain relatively high for most categories, the *Electronics* category shows a higher rate of misclassification, particularly being confused with *Household*. This behavior is likely due to overlapping vocabulary related to usage context and product functionality, which Random Forest models may struggle to separate effectively when operating on sparse TF-IDF features.

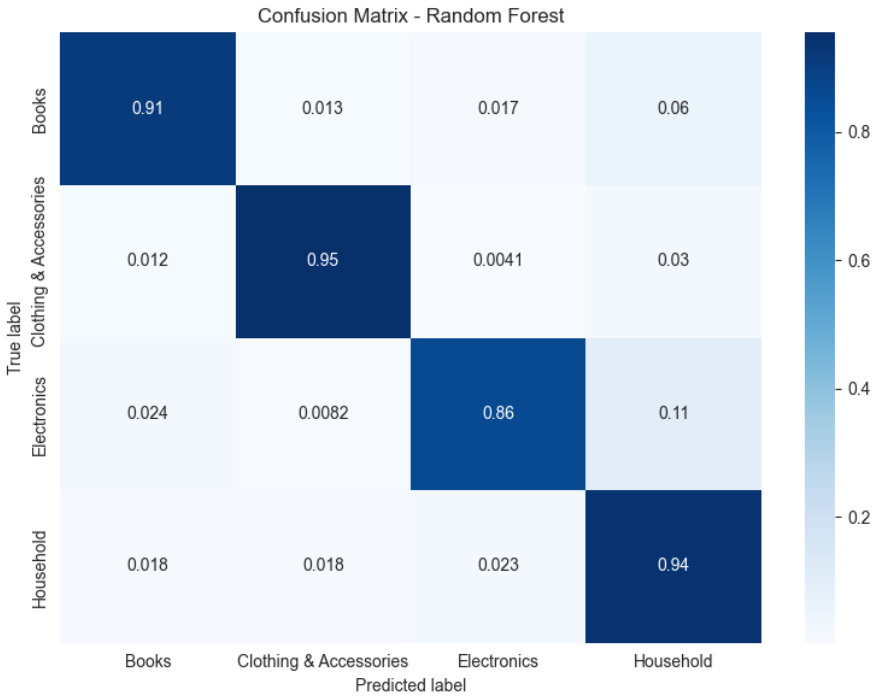


Figure 3.3: Confusion Matrix of Random Forest

Model Comparison

The three classification models evaluated in this project show distinct performance differences when applied to TF-IDF–based product description classification. Overall, the results indicate that linear models outperform the non-linear Random Forest approach for this task.

Among the models, the **Linear Support Vector Machine** achieves the best overall performance, with an accuracy of approximately **96%** and macro-averaged precision and F1-score values of **0.96**. Its strong and consistent results across all categories suggest that margin-based linear separation is particularly effective in high-dimensional sparse feature spaces.

Logistic Regression performs very closely to Linear SVM, achieving an accuracy of approximately **95%** and macro-averaged metrics of **0.95**. While slightly less accurate than Linear SVM, it remains a strong and reliable baseline model, demonstrating stable performance across all categories with only minor class-level confusion.

The **Random Forest** classifier shows comparatively lower performance, with an accuracy of approximately **92%** and macro-averaged metrics around **0.92**. Although it performs reasonably well for some categories, its performance decreases notably for the *Electronics* category, indicating difficulty in separating classes with overlapping vocabulary when using sparse TF-IDF features.

4. Libraries

The project was implemented using a set of widely adopted Python libraries that support data processing, machine learning, visualization, and interactive application development. Each library was selected based on its suitability for text-based classification and information access tasks.

The **Streamlit** library was used to develop the interactive web-based user interface of the application. Streamlit enables rapid prototyping of data-driven applications by allowing Python code to be directly transformed into interactive components such as text inputs, buttons, tables, and plots. In this project, Streamlit was used to create a multi-page interface that allows users to explore the dataset, initiate model training, visualize evaluation results, and perform real-time category prediction. Its session state mechanism was also utilized to persist trained models across different pages of the application.

The **NumPy** library was used as the fundamental numerical computing framework. NumPy provides efficient array-based operations that are essential for handling high-dimensional numerical data produced during text vectorization. In this project, NumPy arrays were used internally by machine learning models and evaluation procedures, particularly in the computation and visualization of confusion matrices.

The **Pandas** library was employed for data loading, cleaning, manipulation, and analysis. Pandas DataFrames were used to represent the dataset in a structured format, making it easier to perform operations such as removing missing values, eliminating duplicate records, grouping data by category, and computing summary statistics. Pandas was also used to organize model evaluation metrics into tabular form, which were then displayed in the user interface for easy comparison.

The **Matplotlib** library was used for data visualization purposes. It was employed to generate bar charts and confusion matrix visualizations that illustrate dataset characteristics and model performance. In particular, Matplotlib was used to visualize category distributions, as well as mean word count and character count statistics before and after text cleaning. These visualizations help to interpret the effects of preprocessing and to analyze classification results at a class level.

The **scikit-learn** library formed the core of the machine learning pipeline. It was used for text feature extraction, model training, and performance evaluation. The TF-IDF vectorization process was implemented using scikit-learn's feature extraction utilities, converting cleaned text into numerical representations suitable for machine learning. Classification models including Logistic Regression, Linear Support Vector Machine, and Random Forest were trained using scikit-learn's implementations. Additionally, scikit-learn provided standardized evaluation metrics such as accuracy, precision, recall, F1-score, and confusion matrices, ensuring consistent and reliable performance assessment across models.

5. How to Run

To run this project, a Python environment is required. The application was developed and tested using Python 3.12.0, and all necessary third-party libraries are listed in the **requirements.txt** file. Before running the application, these dependencies must be installed to ensure that the program executes correctly.

The project directory should contain the following files: app.py, which defines the Streamlit-based user interface and application flow; ml.py, which includes all data preprocessing, model training, and prediction logic; and ecommerceDataset.csv, which contains the e-commerce product descriptions and category labels used in the project. These files work together to provide the full functionality of the system, and all of them must be present in the same directory when running the application.

After ensuring that Python is installed, the required libraries can be installed by running the following command in the project directory:

- **pip install -r requirements.txt**

Once the dependencies are installed, the application can be launched using Streamlit. The following command starts the web-based user interface and runs the application defined in app.py:

- **streamlit run app.py**

After executing this command, the application will open in a web browser, where users can explore the dataset, train classification models, and perform category prediction on new product descriptions through the interactive interface.

6. User Interface

The application provides an interactive web-based user interface developed using Streamlit. The interface is organized into three main pages that guide the user through the workflow of the project: dataset exploration, model training and evaluation, and category prediction. A navigation panel on the left side allows users to switch between these pages, while the main content area dynamically displays visualizations, metrics, and outputs related to the selected stage. This structure enables users to understand the data, observe model performance, and interact with the trained models in a clear and sequential manner.

Page 1: Dataset Overview

The first page displayed when the application is launched focuses on providing an overview of the dataset. As shown in **Figure 6.1** and **Figure 6.2**, this page presents key statistics about the dataset at the top, including the total number of raw samples, the number of samples remaining after data cleaning, and the counts of removed missing and duplicate records. These metrics give an immediate summary of the dataset size and the impact of the preprocessing steps. Below this summary, the distribution of samples across product categories is visualized using a bar chart, allowing users to quickly assess how the cleaned dataset is composed in terms of category balance. The lower section of the page presents text length analyses, where the average word count and average character count are shown for each category both before and after text cleaning. Displaying these two plots side by side makes it easy to compare how the preprocessing pipeline affects textual content across categories and demonstrates the reduction of noise in the product descriptions while preserving meaningful information.

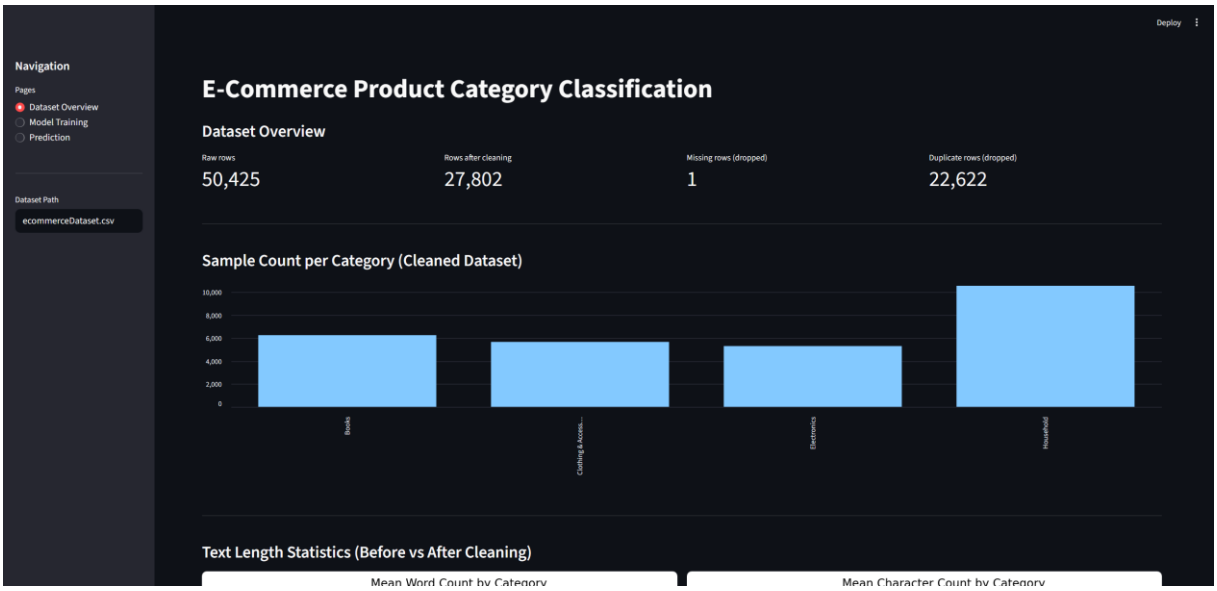


Figure 6.1: Page 1, Dataset Statistics and Category Distribution



Figure 6.2: Page 1, Text Length Statistics

Page 2: Model Training

The Model Training page is designed to allow users to train the text classification models in a controlled and transparent manner and to examine their performance after training. In the first stage, as shown in **Figure 6.3**, the page presents an informational message indicating that model training may take some time, along with a “Train Models” button. This design ensures that model training is explicitly initiated by the user rather than being triggered automatically.

After the training process is started, **Figure 6.4** illustrates how the application provides real-time feedback through a status panel. This panel displays the progress of the training pipeline, including the preparation of TF-IDF features and the training of the Logistic Regression, Linear Support Vector Machine, and Random Forest models. Providing this intermediate feedback improves usability by informing the user about ongoing operations and preventing uncertainty during longer computations.

Once training is completed, **Figure 6.5** shows the evaluation results produced by the models. A table summarizing the performance metrics is displayed, including accuracy, macro-averaged precision, macro-averaged recall, and macro-averaged F1-score for each model. These metrics enable a direct comparison of model performance while treating all categories equally. Below the table, normalized confusion matrices for each model are presented side by side. These visualizations allow for a detailed, class-level analysis of prediction behavior and highlight potential confusions between categories, offering deeper insight into the strengths and limitations of each classification approach.



Figure 6.3: Page 2, Before Model Training

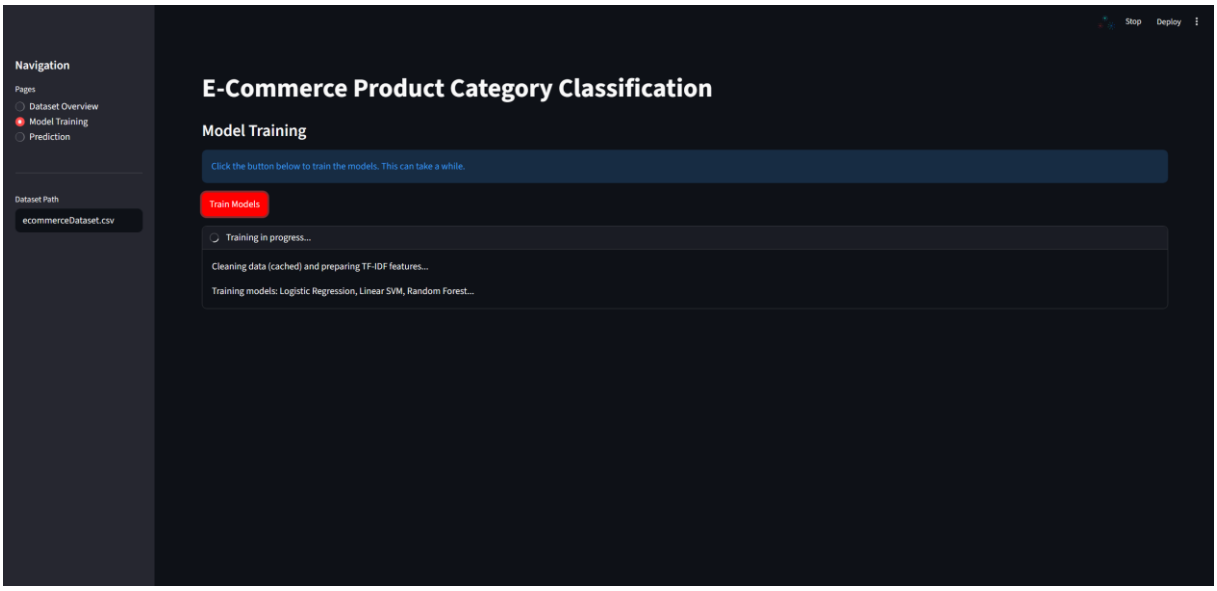


Figure 6.4: Page 2, Model Training in Progress

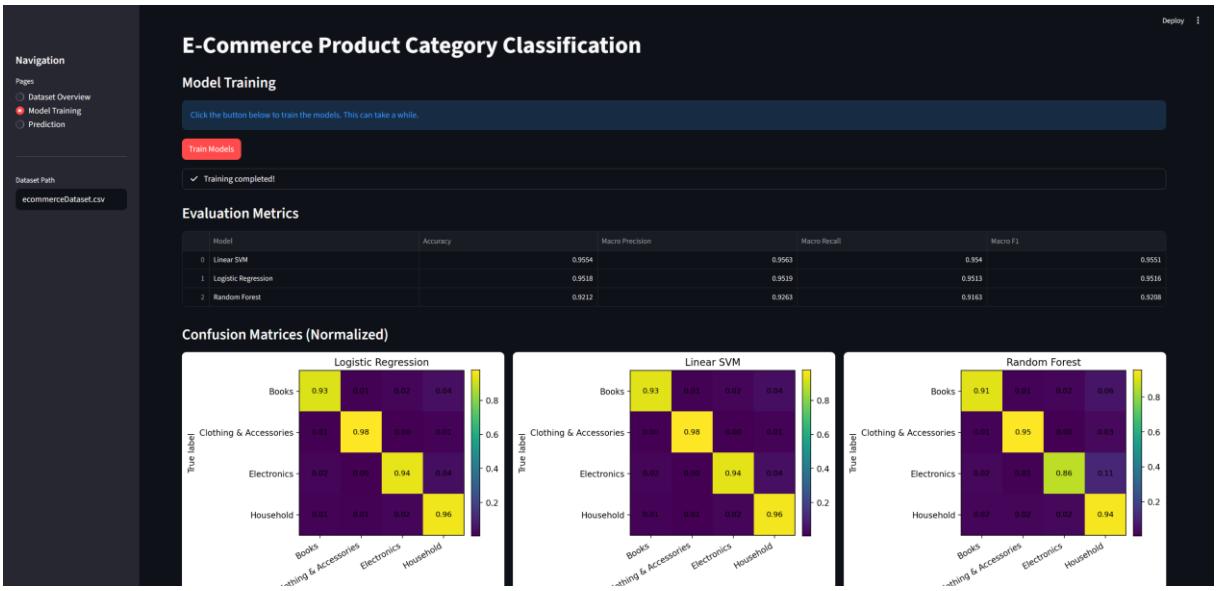


Figure 6.5: Page 2, Training Results

Page 3: Prediction

The Prediction page represents the final stage of the application workflow, where the trained models are used to perform category prediction on unseen product descriptions. As shown in **Figure 6.6**, the page provides a text input area in which the user can enter or paste a product description. A single “Predict” button is used to initiate the classification process, keeping the interaction simple and focused on the core task.

After a product description is submitted, **Figure 6.7** illustrates how the application displays the prediction results generated by each trained model. The predicted category labels from Logistic Regression, Linear Support Vector Machine, and Random Forest are presented separately, allowing users to directly compare the outputs of different classification approaches for the same input text. This side-by-side presentation highlights the consistency or divergence between models and provides insight into how different algorithms interpret the same textual information.

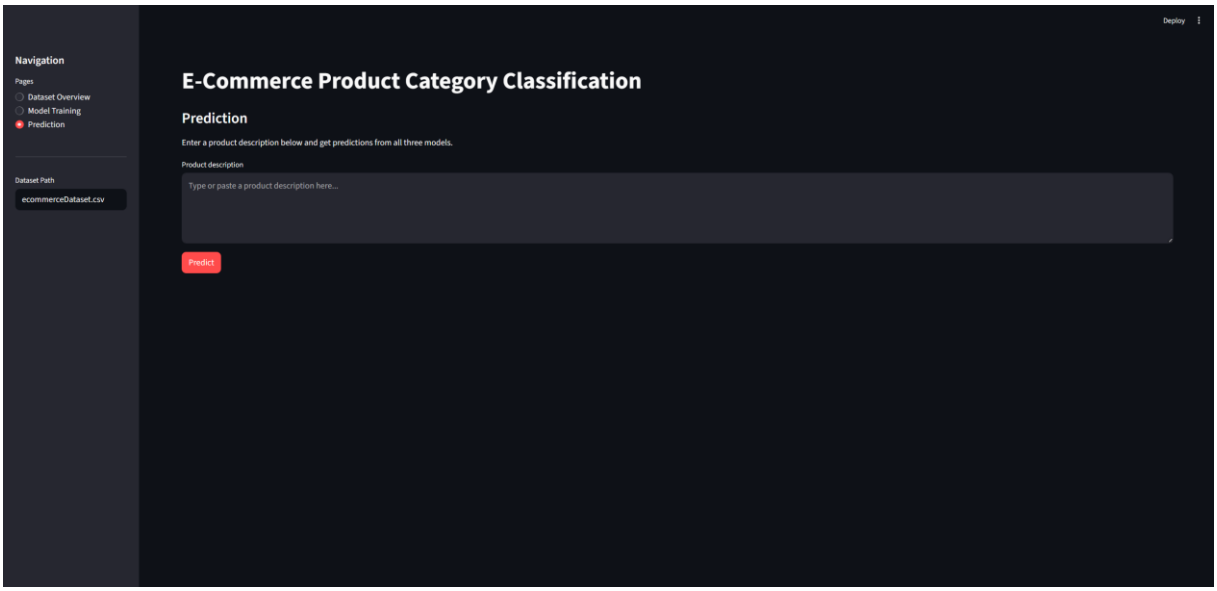


Figure 6.6: Page 3, Prediction Text Area

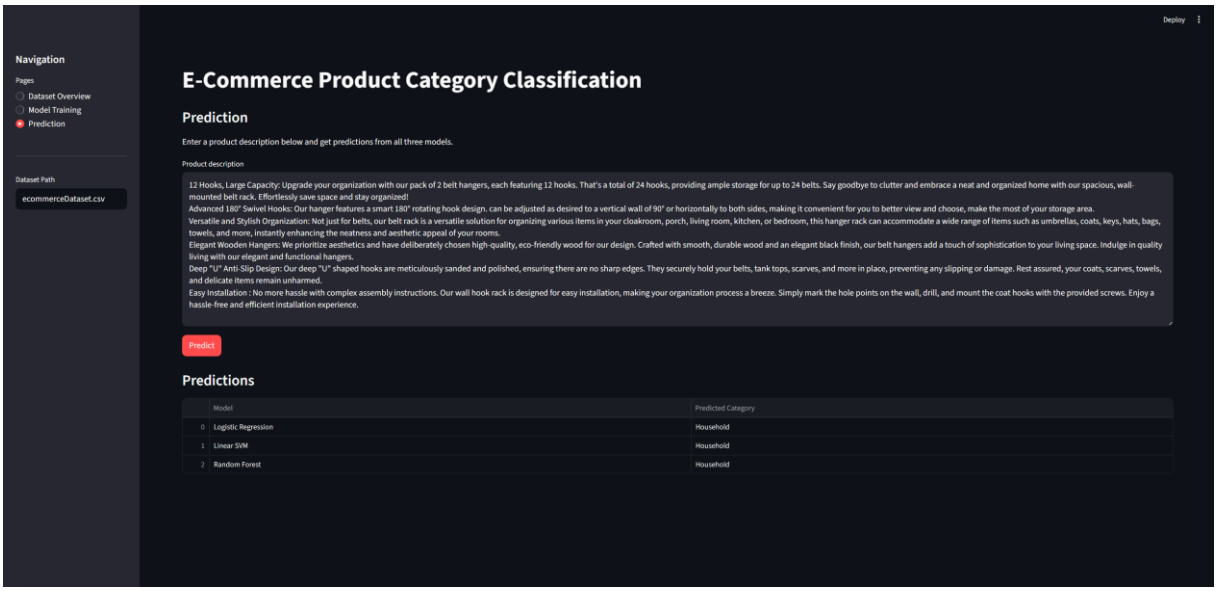


Figure 6.7: Page 3, Prediction Results